Question # 01 (LLO #: 1) [20 Points, 15 Weightage]

Imagine you are a drone operator working for a delivery company. Your drones are currently located at various points in a city, represented by their coordinates on a 2D map. Your central dispatch hub is located at the origin point (0, 0).

At the end of the day, you need to retrieve the **k drones** that are farthest away from the dispatch hub for maintenance, as these drones have traveled the most and are likely in need of servicing.

To determine which drones to retrieve, you use their distances from the dispatch hub. You decide to use a **max-heap** to efficiently find the k farthest drones based on their Euclidean distances from (0, 0).

Drone locations (coordinates): [(1, 3), (2, -2), (5, 8), (0, 1), (7, 6)], **k=2**

K Farthest Drones: [(5, 8), (7, 6)]

Question # 02 (LLO #: 2) [40 Points, 25 Weightage]

You are building a recommendation engine for an e-commerce website. Customers often specify a budget, and the system recommends products whose prices are closest to the budget. The product prices are stored in an AVL tree to ensure efficient insertion, deletion, and retrieval of prices.
The system needs to implement a feature that finds the **closest price** to a given target budget. For example, if a customer has a budget of $250, the system should recommend a product priced nearest to $250.

Write a program to insert product prices into an AVL tree and implement a function to find the product price that is closest to a given target budget **X**.

- **Product prices to insert:** [100, 150, 200, 300, 350, 400]
- **Target budget:** 250
- **Output:** 200

Question # 03 (LLO #: 3) [40 Points, 25 Weightage]

A movie streaming platform tracks movie ratings submitted by users on a scale of 1 to 5. The platform wants to display the ratings in an ordered way based on their frequency.

1. Use a **hash table** to count the number of ratings for each score (1-5):
   - Handle collisions using **separate chaining**.
   - Resize and rehash when the load factor exceeds 0.7.
2. Sort the ratings based on their frequency in **descending order** using **MergeSort**. If two ratings have the same frequency, sort them by the rating value in **descending order**.

Input Example:

Ratings: 5, 4, 3, 5, 5, 4, 2, 4, 5, 3, 3, 3

Output Example:

Sorted Ratings:
1. 5: 4 occurrences
2. 3: 4 occurrences
3. 4: 3 occurrences
4. 2: 1 occurrence